

Advanced Software Architectures

Foundations & Microservices Architecture

Why “Advanced” Architectures Exist (Context Setting)

Core Problem

As software grows:

- Users increase
- Features increase
- Teams increase
- Failures increase

👉 **Architecture exists to manage complexity at scale**



Real-World Analogy

- Small shop → one person does everything
- Medium company → departments
- Large enterprise → independent teams with clear responsibilities



Software architecture mirrors organizational structure

Microservices Architecture – Absolute Basics



What Is Microservices Architecture?

Definition (Beginner-friendly):

Microservices architecture is a way of building software as **many small, independent services**, where each service:

- Does ONE business function
 - Runs independently
 - Communicates over the network
-

🧩 Analogy (Very Important)

“Think of Swiggy or Zomato:

- Order service
- Payment service
- Delivery service
- Notification service

If payment fails, delivery should not crash.”

Each = **microservice**

✗ What It Is NOT

- NOT one big application
 - NOT shared databases everywhere
 - NOT copy-paste services
-

Session 3: Monolith vs Microservices (Comparison)

Aspect	Monolith	Microservice s
Deployment	Single unit	Independent
Scaling	Whole app	Per service
Failure	Entire app	Isolated
Team size	Small	Large teams
Technology	Usually single stack	Polyglot

Tip

Ask:

“If login fails, should payments stop?”

This question **automatically justifies microservices**.

Core Tenets of Microservices

1 Single Responsibility Principle

Each service does **ONE business capability**.

Example:

- User Service → users only
- Order Service → orders only

 Avoid:

- God services
 - Mega APIs
-

2 Independent Deployment

Each service:

- Has its own codebase
- Can be deployed without stopping others

 This enables **CI/CD pipelines**

3 Decentralized Data Management

Each service owns its **own database**.

 Analogy:

“Would HR and Finance share the same Excel sheet?”

4 Technology Independence

- Java service
- Python service
- Node.js service

All coexist.

5 Fault Isolation

Failure in one service should not cascade.

Mini Assessment

MCQs

1. Microservices promote:
 - a) Tight coupling
 - b) Independent deployment
 - c) Shared databases
 - d) Single codebase **b**

Short

- Why is independent deployment important?
-

Inter-Service Communication & Challenges

Inter-Service Communication (Why It's Hard)

Types of Communication

1 Synchronous Communication

- REST
- HTTP
- gRPC

 Caller **waits** for response.

 Analogy:

Phone call — you wait till other person answers

2 Asynchronous Communication

- Message queues
- Events

 Caller does NOT wait.

 Analogy:

WhatsApp message — send and move on

REST-based Communication

REST Basics (Very Simple)

- GET → Read
 - POST → Create
 - PUT → Update
 - DELETE → Remove
-

Sample Interaction

Order Service → calls Payment Service
GET /payment/status/123

Pros

- ✓ Simple
- ✓ Widely supported

Cons

- ✗ Tight coupling
 - ✗ Cascading failures
-

Challenges in Microservices



Major Challenges

1 Network Latency

Remote calls are slow.

✓ Solution:

- Caching
 - Async communication
-

2 Service Discovery

How does one service find another?

✓ Solution:

- Service registry (Eureka, Consul)
-

3 Data Consistency

No shared DB → consistency issues

✓ Solution:

- Eventual consistency
 - Saga pattern
-

4 Observability

Hard to debug distributed systems

 Solution:

- Centralized logging
 - Tracing (Jaeger, Zipkin)
-

Tip

Say this clearly:

“Microservices do NOT reduce complexity.
They **move complexity from code to architecture.**”

Mini Assessment

1. Why is REST risky in microservices?
 2. What is service discovery?
-

Event-Driven Architecture (EDA)

What Is Event-Driven Architecture?

Definition

In EDA, services **react to events** instead of calling each other directly.

Analogy

Doorbell rings → You respond
You don't keep calling the doorbell

Example Event

OrderPlaced
PaymentCompleted
OrderShipped

Components of Event-Driven Systems

1 Event Producer

Creates events.

Example:

- Order Service emits **OrderPlaced**
-

2 Event Broker

Middleman.

Examples:

- Kafka
 - RabbitMQ
-

3 Event Consumer

Listens and reacts.

Example:

- Notification Service sends email
-

Benefits of Event-Driven Architecture

✓ Key Benefits

- ✓ Loose coupling
 - ✓ High scalability
 - ✓ Fault tolerance
 - ✓ Real-time processing
-

Industry Use Cases

- Stock trading systems
 - Payment gateways
 - IoT systems
 - Streaming platforms
-

Design Considerations

🧠 Key Design Points

1 Event Granularity

Events should be:

- Meaningful
- Business-oriented

✗ Avoid technical noise events

2 Idempotency

Consumers must handle duplicate events.

3 Ordering

Events may arrive out of order.

4 Failure Handling

Use:

- Retries
 - Dead-letter queues
-

Microservices + EDA Together

💡 Most modern systems use:

- Microservices for structure
 - Event-driven communication for decoupling
-

Final Assessment (End of Module)

MCQs

1. EDA promotes:
 - a) Tight coupling
 - b) Direct calls
 - c) Loose coupling
 - d) Shared DB☒ **c**
 2. Kafka is:
 - a) Database
 - b) Broker
 - c) UI
 - d) API☒ **b**
-

Short Questions

1. Why are events better than REST in large systems?
 2. What challenges do microservices introduce?
 3. How do microservices and EDA complement each other?
-